ENTREGA 2 / RELATÓRIO CONSOLIDADO

PROJETO INTERAEDU

Da fundação ao produto defendido

Especificação, implementação, testes e governança

Davi Vieira
25 de outubro de 2026

Rede social acadêmica com aplicação Flutter e serviços NestJS. Documento organizado pelos dez passos da entrega, com checklist e apêndice de evidências.

TESTES LOCAIS	RESULTADO	COBERTURA DE LINHAS
97 casos executados	97 aprovados 0 falhas / 0 ignorados	Flutter: 22,00% Backend: 23,94%

Situação do pacote

Candidata à entrega. Os testes e builds locais foram validados. A execução da stack em Docker, a homologação funcional, a proteção da main, a publicação da release ainda exigem conclusão ou aprovação. Davi confirmou o vídeo gravado; falta anexar seu link.

Ferramentas de IA informadas: **Claude e ChatGPT**. O documento distingue relato do responsável, evidência técnica observada e atividades propostas.

GUIA DE LEITURA

Entendendo a Entrega 2

A primeira entrega estabelece a fundação: problema, requisitos, arquitetura e implementação inicial. A segunda transforma essa base em uma solução que pode ser executada, testada, explicada e defendida com evidências. No InteraEdu, essa passagem foi organizada em auditoria, correções, especificação As-Built e preparação da release.

ETAPA	CONTEÚDO NESTE DOCUMENTO
01	Auditoria da Entrega 1 e backlog da sprint final
02	Solução funcional do InteraEdu e critérios de aceite
03	Erros lógicos, regressões e re-especificação
04	PRs, critérios de aceitação e governança
05	Instalação, arquitetura e fluxo de uso
06	ADRs consolidados e defesa das decisões
07	Relato de experiência de Davi Vieira
08	Matriz comparativa: Claude e ChatGPT
09	Relatório ético-técnico em quatro pilares
10	Testes, cobertura e consolidação do PDF
Final	Checklist, cronograma proposto e guia de evidências

Como interpretar as evidências

Verificado: execução ou inspeção registrada. **Preparado:** artefato existente, ainda dependente de execução/aceite. **Pendente:** ação ou comprovação ausente. A ausência de evidência não é preenchida por uma afirmação presumida.

Base da avaliação: repositório alencarrgabriel/intera_edu, commit-base **d04d949** e alterações locais identificadas no manifesto SHA-256. O relatório não declara que esse commit já é a versão final aprovada.

PASSO 1

Auditoria e backlog da sprint final

A auditoria comparou o código e os registros disponíveis com os requisitos de entrega. O documento formal original da Entrega 1 não foi fornecido nesta conversa; portanto, a comparação usa o repositório e a documentação existente como base, sem certificar aderência integral a um enunciado não consultado.

ÁREA	BASE ENCONTRADA	AVANÇO DESTA ETAPA
Testes	36 testes Flutter; backend sem suíte executável instalada.	61 testes backend acrescentados; harness unificado; 97 casos aprovados.
Contratos	Documentação misturava decisões planejadas e comportamento real.	Inventário com 93 entradas de rota, 13 arquivos de DTO e 31 entidades.
Segurança/configuração	Defaults de credenciais, consentimento por body e cache compartilhado.	Segredos via ambiente; identidade por JWT; isolamento de cache e token interno.
Repositório	master sem proteção; artefatos compilados e caches versionados.	Limpeza local e workflows preparados; main/release ainda pendentes.
Execução	Compose antigo, frontend separado e imagem de storage indisponível.	Compose integrado, harness em container e storage com fonte fixa; runtime Docker pendente.

Backlog restante, ordenado por risco

NÍVEL	ITEM / CRITÉRIO DE CONCLUSÃO
P0	Executar Docker e smoke test; exigir todos os serviços saudáveis e relatório sem falhas.
P0	Homologar cadastro/OTP, perfil, publicação, privacidade, arquivo e conversa entre duas contas.
P0	Revisar o commit final, consolidar main protegida e publicar tag apontando para o SHA aprovado.
P1	Anexar o vídeo já gravado e os links de execução e revisão.
P1	Completar comparação empírica das IAs se exigida; revisar relato pessoal e limitações.

PASSO 2

Completando a solução funcional

O exemplo StudyRoom do roteiro é aplicado aqui ao produto real: **InteraEdu**. O problema trabalhado é a aproximação de universitários para compartilhar conhecimento, encontrar pessoas por interesses e manter interações acadêmicas em um ambiente institucional.

FLUXO	IMPLEMENTAÇÃO PRESENTE	ACEITE FUNCIONAL A REGISTRAR
Acesso institucional	Registro, OTP, login, refresh e recuperação de senha.	Cadastro com domínio permitido; rejeição de domínio/OTP inválido; sessão e logout.
Perfil e conexões	Dados acadêmicos, habilidades, privacidade e solicitações de conexão.	Editar perfil; solicitar/aceitar/rejeitar; testar acesso de outra conta.
Feed e publicações	Feed local/global, posts, comentários, reações e favoritos.	Publicação persiste; contadores coerentes; reação/favorito isolado por usuário.
Conteúdo acadêmico	Grupos, disciplinas, materiais, stories, tags e menções.	Demonstrar apenas os recursos efetivamente validados na versão exibida.
Mensagens	Chats, membros, texto, anexos e Socket.IO.	Enviar/receber entre clientes; impedir acesso por não membro; testar reconexão.

Recorte demonstrável

O fluxo principal sugerido é: entrar com conta de demonstração, consultar perfil, publicar no feed, conectar-se a outro estudante e trocar uma mensagem. Esse recorte oferece uma sequência compreensível do problema até o valor entregue, sem depender de apresentar todas as telas.

Estado de homologação

Os testes locais confirmam comportamentos delimitados. O vídeo foi confirmado por Davi, mas não disponibilizado para análise. Não há registro e2e nesta rodada comprovando o fluxo completo com banco, storage e dispositivos reais. Essa etapa permanece no backlog de aceite.

Evidências: lib/core/router/app_router.dart; serviços e controllers do backend; docs/entrega/sdd-as-built.md; docs/entrega/roteiro-video.md.

PASSO 3

Refinamento por testes e re-especificação

O refinamento transforma uma divergência observada em regra explícita, correção e regressão verificável. Os casos abaixo foram tratados nesta preparação; não são propostas apresentadas como já implementadas.

ID	ERRO / DIVERGÊNCIA	CONTRATO REFINADO E VERIFICAÇÃO
RE-07	Consentimento aceitava user_id do corpo.	O titular vem de CurrentUser/JWT. Testes HTTP rejeitam ausência de token e ignoram identidade forjada.
RE-08	Cache de feed compartilhava reação/favorito entre usuários da mesma instituição.	A chave inclui o usuário. Teste exige chaves distintas para Alice e Bob.
RE-09	Prefixo semelhante podia ser encaminhado como rota válida.	Rota deve ser prefixo exato ou subcaminho. Integração exige 404 para /users-evil.
RE-10	Notificação interna não exigia credencial de serviço.	X-Internal-Token obrigatório; ausência, token errado ou configuração ausente retornam 401.
RE-11	Readiness era estático e alguns healthchecks faltavam/eram protegidos.	Liveness público; readiness consulta quatro serviços e retorna 503 em indisponibilidade.
RE-12	OTP podia aparentar envio sem provedor ou após erro externo.	Envio indisponível retorna 503; log de OTP exige opção explícita de desenvolvimento.
RE-13	JavaScript compilado coexistia com TypeScript, afetando resolução/cobertura.	Cópias geradas removidas; Jest prioriza TS. Nova execução mede os fontes atuais.

Ciclo adotado

Reproduzir o comportamento → especificar esperado → implementar o ajuste → executar a regressão → atualizar o As-Built → revisar o diff. Os testes não substituem revisão humana nem justificam alterar um requisito só para acomodar um defeito.

Evidências: backend/test/auth.test.js; domain.test.js; http-integration.test.js; gateway-integration.test.js; notifications.test.js; otp.test.js. Os refinamentos RE-01 a RE-06 permanecem no relatório histórico de 12/10.

PASSO 4

Governança: comprovado e pendente

A governança precisa ser demonstrada por registros, não apenas descrita como intenção. A consulta pública ao GitHub em 25/10/2026 e o histórico local permitem as conclusões abaixo.

VERIFICAÇÃO	EVIDÊNCIA / CONCLUSÃO
Branches e PRs históricos	O histórico inclui merges, como o PR #11 no commit 5f1b26f. Isso comprova uso de PR, mas não comprova revisão independente ou aprovação qualificada.
PRs abertos	Consulta pública: zero PRs abertos naquele momento. Deve ser revalidado antes da publicação.
Branch principal	master é a padrão e está sem proteção. Não há main na consulta registrada.
Release oficial	Nenhuma release existente na consulta. A tag estável ainda não foi publicada.
Sprint/board/revisores	Não há evidência suficiente para atribuir datas de sprint, reuniões, issues ou revisão por outro integrante.

Critérios de aceitação para integrar a candidata

1. Harness e build aprovados no commit revisado.
2. Configuração Docker e smoke test aprovados em runtime real.
3. Contratos e documentação compatíveis com os fontes.
4. Nenhum segredo ou artefato temporário no conjunto versionado.
5. Limitações conhecidas registradas e fluxos de demonstração homologados.
6. Aprovação humana identificada antes da tag oficial.

Mecanismos preparados

O workflow de validação executa harness, stack Docker e smoke test. O de publicação exige execução na main, SHA aprovado correspondente, branch protegida e zero PRs abertos. As regras de proteção e o environment de aprovação ainda precisam ser configurados no GitHub.

Responsabilidade

Somente Davi Vieira foi informado como integrante. Revisão independente exige um revisor real; não se presume uma equipe ou aprovação que não esteja documentada.

Evidências: reports/release/github-state.json; .github/workflows/validate-release.yml; publish-release.yml; docs/entrega/checklist-release.md.

PASSO 5 / EXECUÇÃO

README final e instalação

O README foi atualizado para concentrar ambiente, variáveis, execução, testes e evidências. Os comandos abaixo partem da raiz do repositório. O ambiente é de demonstração, com sincronização TypeORM em desenvolvimento.

Pré-requisitos

Containers: Docker com Compose v2.20+ e Python 3.10+ para configuração. Execução local: Flutter 3.38.10, Dart compatível com ^3.10.0, Node 20 como referência de CI/containers e Python 3.10+. Android também requer JDK 17, SDK e emulador/aparelho.

Configuração e aplicação

```
python .scripts/init-env.py
docker compose --env-file backend/.env up --build -d --wait
```

O gerador cria backend/.env com segredos aleatórios e preserva arquivo existente. Configurar RESEND_API_KEY/RESEND_FROM para OTP real e S3_PUBLIC_ENDPOINT para um endereço alcançável pelo dispositivo. O código usa Resend; variáveis SMTP não configuram o envio atual.

Frontend: **localhost:8080**. API: **localhost:3000/api/v1**. Readiness: **/api/v1/health/ready**. Socket.IO: porta 3004, namespace /ws.

Harness e diagnóstico

```
npm --prefix backend ci --no-audit --no-fund
flutter pub get --enforce-lockfile
python .scripts/run-tests.py

docker compose -f compose.test.yaml run --build --rm harness
node .scripts/smoke-test.mjs
```

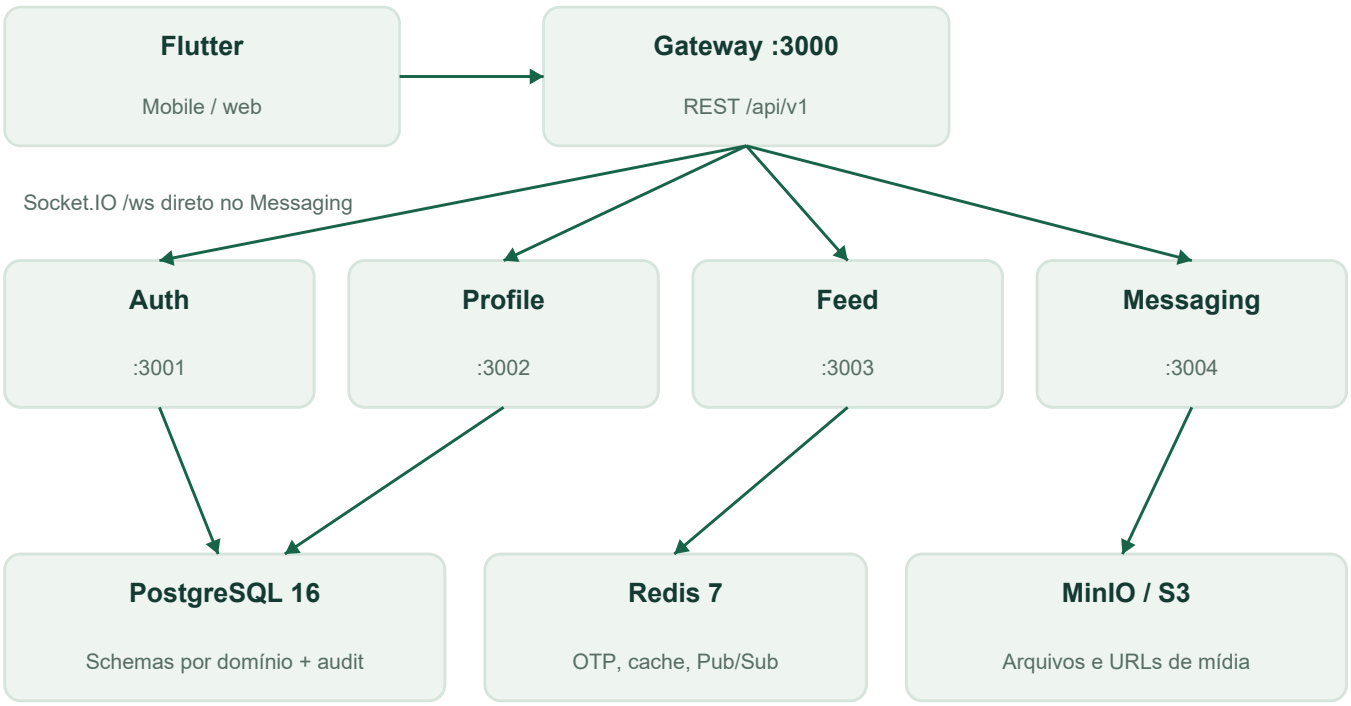
A primeira linha do segundo bloco é a alternativa de testes em container. O smoke test exige a aplicação já iniciada. Logs e resultados são gravados em reports/release. O harness retorna código diferente de zero quando uma etapa falha.

Resultado observado

Os arquivos Compose passaram na validação oficial de configuração; não foram executados em engine Docker neste computador. O build web e os seis builds backend foram executados localmente com sucesso.

PASSO 5 / ARQUITETURA

Arquitetura e fluxo da aplicação



O cliente Flutter usa repositórios e um cliente HTTP para o gateway. Os serviços validam autenticação e regras de domínio. As conexões em tempo real usam diretamente o serviço de mensagens. A persistência de desenvolvimento é um PostgreSQL compartilhado por schemas.

FLUXO	SEQUÊNCIA EFETIVA
Cadastro	Email institucional → OTP → token temporário → credencial/consentimento → tokens → evento de cadastro → perfil. O evento usa Pub/Sub best-effort.
Publicação	Tela → notifier/repositório → API → serviço de posts → banco/storage → resposta → atualização de estado/feed. Cache pode reter dados por até 60 segundos.
Mensagem	Cliente autenticado → chat → verificação de membro → persistência → evento message:new para a sala. O fluxo entre dispositivos ainda precisa de homologação.

Limites arquiteturais: há consultas SQL entre schemas no perfil/exportação; não existe isolamento físico completo de bancos. Objetos MinIO usam política pública de leitura. Isso exige evolução antes de tratar anexos como privados.

Evidências: SDD As-Built; inventário de 93 entradas HTTP, 13 arquivos de DTO e 31 entidades. As assinaturas e regras detalhadas permanecem nos documentos técnicos anexos ao repositório.

PASSO 6 / DECISÕES 1-3

ADRs consolidados

Os ADRs são uma consolidação retrospectiva das escolhas observadas. Não são atas de decisões passadas nem comprovam participação ou aprovação de pessoas não registradas.

ADR-001 | Flutter e separação em camadas

Contexto: A aplicação reúne telas com sessão, rede e entidades compartilhadas.
Decisão: Organizar core, domain, data e presentation; usar notifiers/Provider.
Alternativa: Concentrar rede e estado diretamente nas telas.
Defesa e custo: Favorece testes e contratos, mas aumenta arquivos, mocks e sincronização entre camadas. Para um primeiro mobile, essa complexidade precisa ser administrada.

Evidências: lib/; test/notifiers; test/network; test/widgets.

ADR-002 | NestJS por domínio e gateway

Contexto: Autenticação, perfil, feed e mensagens têm responsabilidades distintas.
Decisão: Separar serviços e compartilhar JWT/Redis/DTOs em um pacote comum.
Alternativa: Monólito modular.
Defesa e custo: Separação facilita localizar responsabilidades; aumenta processos, configuração, rede e modos de falha. Um monólito seria mais simples operacionalmente para um MVP pequeno.

Evidências: backend/package.json e módulos dos serviços.

ADR-003 | PostgreSQL e Redis

Contexto: A solução precisa de persistência relacional, TTL e comunicação de eventos.
Decisão: Schemas por domínio; Redis para OTP, cache e Pub/Sub.
Alternativa: Banco sem fronteiras ou mensageria durável com confirmação.
Defesa e custo: O isolamento por schema é parcial e há consultas cruzadas. Pub/Sub não garante entrega durável; cadastro/perfil precisam de testes integrados e recuperação de falhas.

Evidências: shared/database; shared/redis; init-db.sql; events.subscriber.ts.

PASSO 6 / DECISÕES 4-6

Defesa das decisões de entrega

ADR-004 | JWT e renovação limitada

Contexto: tokens expiram e requisições podem receber 401.
Decisão: backend verifica o token; cliente restaura estado a partir de claims e faz no máximo uma repetição após refresh.
Alternativa: renovar indefinidamente ou pedir login em toda expiração.
Defesa: evita ciclos e preserva erros de credenciais. A inspeção local de claims não substitui autorização criptográfica no servidor.

Evidências: ApiClient, AuthNotifier, estratégia JWT e testes de sessão.

ADR-005 | Compose e storage compatível com S3

Contexto: reproduzir aplicação, banco, cache e arquivos.
Decisão: Compose com imagens-base por digest; MinIO construído de commit oficial e checksum fixos.
Alternativa: instalação manual ou serviços gerenciados.
Defesa: descreve entradas do build e reduz variação. O primeiro build é pesado e a execução Docker ainda precisa de evidência. Objetos públicos e synchronize limitam uso com dados reais.

Evidências: compose.yaml; Dockerfiles; deploy/images.lock.json; storage/Dockerfile.

ADR-006 | Harness único e evidência por execução

Contexto: testes isolados e resultados históricos dificultam defender a versão entregue.
Decisão: um comando compila backend, roda Jest/Flutter e gera JSON, JUnit, LCOV e hashes dos fontes.
Alternativa: comandos manuais sem consolidação.
Defesa: torna o procedimento repetível e auditável. A aprovação mede os casos executados; cobertura e e2e continuam critérios separados.

Evidências: run-tests.py; release-report.py; reports/release/run.json.

Síntese para a defesa oral

Justifique cada escolha pelo problema resolvido, explicite o custo e a alternativa descartada, e mostre a evidência correspondente. Evite apresentar uma tecnologia como superior em qualquer contexto.

PASSO 7

Relato de experiência

Responsável informado: **Davi Vieira**. Ferramentas utilizadas: **Claude e ChatGPT**. Não foram informados outros integrantes ou uma divisão de tarefas entre pessoas; por isso, o relato preserva a experiência individual e não inventa dinâmica de grupo.

> *“A maior dificuldade foi por desenvolver uma aplicação mobile, nunca tinha feito e achei bem mais complexo de se desenvolver.”*

Relato consolidado para revisão do autor

O InteraEdu foi minha primeira experiência de desenvolvimento de uma aplicação mobile. A dificuldade principal foi lidar com uma complexidade maior do que eu esperava. O trabalho exigiu compreender não só as telas, mas também o estado da interface, a sessão, a comunicação com o backend e a configuração necessária para executar o sistema.

Utilizei Claude e ChatGPT durante o projeto. A preparação da entrega evidenciou que uma funcionalidade descrita ou implementada ainda precisa de verificação prática. Testes, contratos e documentação ajudam a transformar uma solução que parece correta em uma solução cujo comportamento pode ser explicado e conferido.

A principal lição técnica é organizar desde o início os critérios de aceitação e as evidências. A principal lição de governança é manter claro o que está implementado, o que foi testado e o que ainda depende de revisão ou homologação. A IA pode ajudar na elaboração e no refinamento, mas a responsabilidade pela versão entregue permanece com o responsável pelo projeto.

Produtividade e trabalho coletivo

Foram produzidos testes, relatórios e documentação repetível nesta etapa. Não há medição de horas que permita declarar um percentual de produtividade. Também não há registro suficiente para avaliar distribuição de trabalho ou revisão entre pares. Essas lacunas devem permanecer explícitas na defesa.

Validação pessoal

O trecho entre aspas reproduz o relato fornecido por Davi. Os demais parágrafos são uma síntese editorial baseada nesse relato e nos artefatos observados; precisam de leitura e aceite do autor antes da submissão.

PASSO 8

Matriz comparativa das ferramentas de IA

A comparação considera **Claude** e **ChatGPT**, as duas ferramentas informadas por Davi. Não foi realizado um experimento controlado com a mesma tarefa, modelos, prompts e tempo. A matriz compara a evidência disponível, sem inventar notas ou superioridade entre produtos.

CRITÉRIO	CLAUDE	CHATGPT / ASSISTÊNCIA DESTA ETAPA
Uso no projeto	Uso confirmado pelo responsável. Interface/modelo e tarefas específicas não registrados.	Uso confirmado; esta conversa registra inspeção, alterações, testes e documentação.
Aderência à especificação	Sem conjunto de prompts/patches para avaliação independente.	Foram comparados contratos e código; diferenças foram incorporadas ao As-Built.
Correção e testes	Não há testes atribuíveis exclusivamente à ferramenta.	61 testes backend acrescentados; execução consolidada de 97 casos. Aprovação depende dos testes e da revisão, não da autoria da sugestão.
Segurança e limites	Não mensurados nesta documentação.	Correções verificadas de identidade, cache e token interno; limites de Docker, cobertura e release registrados.
Produtividade	Sem medição de tempo ou intervenções.	Artefatos produzidos são observáveis; ganho percentual não medido.
Limitação da comparação	Ausência de logs impede atribuição detalhada.	Maior disponibilidade de evidência não demonstra superioridade geral sobre Claude.

Protocolo para uma comparação empírica futura

Aplicar às duas ferramentas o mesmo recorte de código e critério de aceite, em cópias isoladas. Registrar modelo/versão, prompt, tempo, intervenções e patch. Rodar a mesma suíte e avaliar aderência, correção, casos de borda, segurança e clareza. Só então atribuir notas ou concluir sobre ganho de produtividade.

Se o enunciado exigir um benchmark empírico entre ferramentas, esta matriz descritiva não substitui essa execução.

PASSO 9

Relatório ético-técnico: quatro pilares

Os quatro pilares adotados organizam os riscos do uso de IA e os controles do projeto. Eles não representam certificação jurídica, auditoria de segurança completa ou aprovação humana já concluída.

1. Confiabilidade, alucinação e segurança

Risco: sugestões plausíveis podem divergir do contrato, omitir autorização ou produzir testes que não exercitam o fonte atual.
Controle aplicado: inspeção do diff, regressões de identidade/cache, remoção de cópias compiladas e nova execução de testes.
Limite: cobertura parcial e integração isolada não garantem ausência de defeitos no produto inteiro.

2. Privacidade e confidencialidade

Risco: exposição de .env, tokens, emails, mensagens e dados acadêmicos em prompts ou logs.
Controle aplicado: segredos gerados localmente, arquivos de ambiente ignorados, dados sintéticos nos testes e OTP em log somente por opção explícita de desenvolvimento.
Pendente: revisão do histórico e da política de anexos, hoje com leitura pública no storage.

3. Propriedade intelectual e autoria

Risco: tratar código gerado como automaticamente original ou ignorar licenças de dependências e vendor.
Controle aplicado: identificação de Claude/ChatGPT e do responsável; origem fixa do MinIO e licença preservada na imagem.
Pendente: auditoria de licenças e decisão do titular sobre a licença do projeto. Não se presume titularidade a partir da geração por IA.

4. Homologação humana e responsabilização

Risco: publicar como estável algo sem revisão, inventar aprovação ou apresentar métricas fora de contexto.
Controle preparado: checklist, checks de CI, referência ao SHA aprovado e exigência de main protegida na publicação.
Pendente: revisão/aceite identificados, configuração real das regras e demonstração funcional. Davi continua responsável pelo conteúdo entregue.

PASSO 10 / EXECUÇÃO

Relatório de testes: 100% pass

Na execução consolidada de **25/10/2026, das 12:25:46 às 12:28:30** (America/Sao_Paulo), todos os casos coletados passaram. O commit-base é d04d949; alterações testadas são identificadas por hash no manifesto. Esta aprovação não equivale à homologação de uma release publicada.

SUÍTE	TOTAL	APROVADOS	FALHAS	IGNORADOS
Flutter	36	36	0	0
Backend	61	61	0	0
Total	97	97	0	0

Etapas automatizadas

ETAPA	RESULTADO E ESCOPO
Build backend	Sucesso nos seis workspaces: shared, gateway, auth, profile, feed e messaging.
Testes backend	6 suítes Jest; unidade e integração HTTP com servidores locais e dependências isoladas.
Testes Flutter	Modelos, rede, sessão, notifiers e widgets; sem execução em aparelho real.
Build web adicional	Compilação release concluída; cache local de dependências foi isolado após arquivos ausentes no cache global.
Compose	Configuração válida no validador oficial; nenhum container foi iniciado nesta máquina.

Cobertura não é taxa de aprovação



A taxa de aprovação é **100% dos 97 casos executados**. A cobertura de linhas é **22,00% no Flutter** (1.285/5.841) e **23,94% no backend** (480/2.005). Esses percentuais medem dimensões diferentes e devem ser apresentados juntos.

PASSO 10 / COBERTURA

Unidades críticas e limites da evidência

UNIDADE BACKEND	LINHAS COBERTAS	COBERTURA
Autenticação	81/123	65,85%
OTP	50/53	94,33%
Serviço de proxy	19/23	82,60%
Controller de proxy	36/55	65,45%
Guard JWT	11/11	100,00%
Estratégia JWT	7/7	100,00%
Conexões	27/67	40,29%
Posts/feed	46/161	28,57%
Chats	32/95	33,68%
Notificações	13/16	81,25%

No Flutter, o relatório detalha também ApiClient (52,21%), AuthNotifier (66,15%) e FeedNotifier (62,96%). Os números completos estão nos arquivos LCOV e no resumo por arquivo.

O que foi realmente integrado

Os testes HTTP iniciam aplicações Nest reais, exercitam validação de DTO, autenticação JWT e o proxy para um servidor HTTP local. Repositórios e Redis são isolados por mocks em testes de domínio. Isso permite verificar contratos sem depender da infraestrutura externa.

O que ainda não foi homologado

PostgreSQL, Redis, MinIO, Resend, criação de perfil por evento, uploads e conversas entre dispositivos não foram exercitados como uma stack completa nesta rodada. A medição backend exclui bootstrap, módulos de DI, entidades, DTOs e interfaces; inclui serviços/controllers sem testes, reduzindo a chance de ocultar lacunas.

Evidências: reports/release/relatorio.md; summary.json; junit.xml; backend-tests.json; flutter-events.jsonl; backend-coverage/coverage-summary.json; arquivos LCOV.

CHECKLIST FINAL

Itens obrigatórios e situação real

ITEM	SITUAÇÃO	EVIDÊNCIA / PRÓXIMO PASSO
Auditoria e backlog	Preparado	Auditoria no Passo 1; backlog de fechamento priorizado.
Solução funcional	Parcial	Implementação existente; homologar o fluxo completo e registrar resultado.
Testes e re-especificação	Verificado localmente	97 casos aprovados; contratos refinados e regressões documentados.
Governança e sprints	Parcial	PRs históricos e consulta atual; faltam regras, revisão e registros de sprint.
README e arquitetura	Preparado	Guia, diagrama, variáveis e comandos atualizados.
ADRs	Preparado	Seis decisões consolidadas com alternativas, custos e evidências.
Relato de experiência	Preparado	Davi Vieira; síntese pessoal sujeita à revisão do autor.
Comparativo de IA	Descritivo	Matriz Claude/ChatGPT; benchmark controlado não realizado.
Ética: quatro pilares	Preparado	Riscos, controles e lacunas identificados.
Relatório/PDF e cobertura	Verificado localmente	Resultados reais, LCOV/JUnit/JSON e este relatório.
main protegida e release	Pendente	Migrar/consolidar master, proteger main, aprovar SHA e publicar tag.
Docker reproduzível	Parcial	Configuração e referências verificadas; executar containers e smoke test.
Vídeo de 3-5 minutos	Gravado	Confirmado por Davi; link e duração real a informar.

Condição para declarar a entrega concluída

Os itens preparados e os testes locais estão disponíveis. A conclusão integral exige fechar as pendências de execução, aprovação e evidência, especialmente release, governança e vídeo. Este PDF não converte pendências em resultados obtidos.

APÊNDICE A

Cronograma da sprint e critérios de saída

O quadro é um **plano proposto de fechamento**, não um calendário retrospectivo de trabalho realizado. D1 a D5 são sessões/dias relativos a agendar por Davi; a duração deve ser ajustada ao resultado da homologação.

DIA	ATIVIDADE	CRITÉRIO DE SAÍDA
D1	Revisar auditoria, backlog e As-Built; instalar/validar Docker.	Escopo e limitações aceitos; ambiente inicia sem falhas de healthcheck.
D2	Executar smoke e fluxos de cadastro, perfil, post, conexão, upload e conversa.	Esperado/obtido registrado; erros reproduzíveis viram issues e testes.
D3	Corrigir falhas e reexecutar harness; revisar ADRs, IA e ética.	Checks aprovados na versão candidata e documentação alinhada.
D4	Revisar PR/commit, consolidar main, configurar proteção e publicar release.	SHA aprovado, regras ativas, zero PRs pendentes e URL da release.
D5	Anexar vídeo já gravado e conferir pacote final.	Vídeo de 3-5 min, PDF revisado, links acessíveis e evidências do mesmo commit.

Registros históricos efetivamente disponíveis

17/10/2026: relatório de 36 testes Flutter aprovados; backend então bloqueado pela ausência de Jest/testes.
25/10/2026: novo harness, 61 testes backend, 97 casos aprovados no total, build web/backend e validação de configuração Compose.
GitHub na consulta de 25/10: master sem proteção, zero PRs abertos e nenhuma release.

Como comprovar a sprint

Para cada item, registrar responsável, critério de aceite, issue, branch/PR, decisão de revisão e evidência. O cronograma proposto passa a ser histórico apenas quando essas ações ocorrerem e forem registradas. Não criar atas, issues concluídas ou aprovações retroativas fictícias.

APÊNDICE B

Guia de evidências e pacote de submissão

ID	ARTEFATO	FINALIDADE
E01	README.md	Instalação, execução e visão de entrega.
E02	docs/entrega/sdd-as-built.md	Arquitetura e limitações efetivas.
E03	contratos-as-built.md / modelos-as-built.md	Rotas, DTOs e entidades extraídos dos fontes.
E04	reports/release/run.json / source-sha256.json	Data, comandos, códigos de saída e fontes testados.
E05	summary.json / junit.xml / logs de teste	Contagem de casos e resultado verificável.
E06	flutter-lcov.info / backend-coverage/	Cobertura geral e por unidade.
E07	hygiene.json / artifact-verification.json	Scan de padrões e conferência dos artefatos.
E08	github-state.json / compose-validation.json	Estado público consultado e validação de configuração.
E09	deploy/images.lock.json	Digests das imagens-base; não prova execução.
E10	checklist-release.md / roteiro-video.md	Aceite e roteiro para completar a demonstração.

Os nomes abreviados de E03/E10 ficam em docs/entrega; E05-E08 ficam em reports/release. Anexar os arquivos digitais ao repositório/release mantém os detalhes auditáveis sem inserir milhares de linhas de log no PDF.

Itens ainda a anexar

URL da release e SHA final; evidência de proteção da main; checks de CI do mesmo commit; revisão humana identificada; relatório Docker/smoke e fluxos homologados; link do vídeo. Na apresentação, mostrar resultados ao vivo ou identificar claramente um corte/registro anterior.

Referências verificáveis

Repositório InteraEdu: github.com/alencarrgabriel/intera_edu
MinIO: revisão oficial usada pelo Dockerfile de storage

As referências de código e resultados correspondem à candidata local identificada nos manifests. Novos commits, testes ou homologações exigem atualizar as evidências e, quando necessário, regenerar o PDF.